



Nombre:

Yinell Abigail Echavarria Lorenzo

Matrícula:

2026-0301

Maestro:

Carlos Antonio Pichardo Viuque

Grupo:

No.3

**El Lenguaje de la Automatización: La Lógica
Matemática como Fundamento de la Programación
en Mecatrónica**

Introducción

La mecatrónica es la unión de la mecánica, la electrónica y la programación para crear sistemas automáticos inteligentes. En esta carrera, el software es como el sistema nervioso que recibe los datos de los sensores y les dice a los motores y actuadores lo que tienen que hacer. Pero antes de ponernos a escribir código en C, C++ o Python, necesitamos una base sólida para ordenar nuestras ideas y asegurarnos de que el programa no falle. Esa base es la lógica matemática.

Tomando como guía el Capítulo IV del libro *Matemáticas para la Computación* de José Alfredo Jiménez Murillo, en este ensayo voy a explicar cómo temas como las proposiciones, las tablas de verdad, las demostraciones y los predicados no son solo teoría aburrida de matemáticas. Al contrario, son las herramientas que usamos todos los días para diseñar, limpiar y mejorar el código que hace que los robots y las máquinas de una fábrica funcionen de manera correcta y segura.

1. El Concepto de Proposición en el Mundo Real

Una proposición es simplemente una frase que puede ser verdadera (V) o falsa (F), sin puntos medios. En programación para mecatrónica, estas proposiciones dejan de ser palabras y se convierten en estados reales de los sensores que conectamos a un microcontrolador.

Cuando combinamos estas frases usando la negación, la conjunción o la disyunción, creamos proposiciones compuestas. En lenguajes como C o C++, esto lo hacemos usando los operadores lógicos `!`, `&&` y `||`. Por ejemplo, podemos definir dos situaciones en una máquina:

- p : El sensor óptico detecta que hay una pieza en la banda.
- q : El brazo robótico está en su posición inicial.

Si juntamos ambas $(p \wedge q)$, creamos una condición compuesta: la máquina solo va a arrancar si la pieza está en la banda Y el robot está listo.

Condicional y Bicondicional

El libro de Jiménez Murillo también explica dos conectores lógicos que usamos para controlar el flujo de un programa:

- El Condicional $(\rightarrow)$: Es el famoso "si pasa esto, entonces haz aquello" $(p \rightarrow q)$. Es exactamente lo mismo que usamos al escribir un `if` en el código. Por ejemplo: Si el motor se calienta demasiado (p), entonces apaga la máquina (q).
- El Bicondicional $(\leftrightarrow)$: Esto significa que una cosa pasa "si y solo si" pasa la otra $(p \leftrightarrow q)$. En mecatrónica sirve para sincronizar procesos. Por ejemplo, el pistón de rechazo se activa si y solo si la pieza es defectuosa.

2. Las Tablas de Verdad en la Programación

Las tablas de verdad nos sirven para ver qué pasa con un circuito o un programa bajo todas las combinaciones posibles de los sensores. El libro las clasifica en tres tipos, y cada una tiene un significado muy práctico para nosotros:

- Tautología: Es cuando el resultado siempre es verdadero, pase lo que pase. En nuestro caso, una tautología es la lógica detrás de los bucles infinitos que mantienen funcionando a un microcontrolador, como el `while(true)` o el `void loop()` en Arduino.
- Contradicción : Es cuando el resultado siempre da falso. En programación, una contradicción es un error grave (*bug*). Si escribimos una condición que matemáticamente es una contradicción (por ejemplo, pedirle al código que un motor gire a la izquierda y a la derecha al mismo tiempo: $m \wedge \neg m$), esa parte del código nunca se va a ejecutar y la máquina se va a quedar trabada.
- Contingencia : Es cuando el resultado puede ser verdadero o falso dependiendo de la situación. Esto es la automatización real: el robot tiene que tomar decisiones diferentes según lo que vaya encontrando en su camino.

3. Inferencia y Equivalencia Lógica para Mejorar el Código

La *inferencia lógica* es sacar conclusiones válidas a partir de datos que ya sabemos que son ciertos (usando reglas como el Modus Ponens). En mecatrónica, esto sirve para que las máquinas detecten fallas solas. Si el programa sabe que "si la corriente pasa de 10 amperios, el motor se va a quemar", y el sensor mide 12 amperios, el sistema infiere el peligro y apaga todo para proteger el hardware.

Por otro lado, la *equivalencia lógica* nos muestra que dos condiciones que se escriben diferente pueden significar exactamente lo mismo. Usando las Leyes de De Morgan y otras reglas que vienen en el libro, podemos simplificar líneas de código que son muy largas o confusas.

Al hacer el código más corto y directo, el microcontrolador gasta menos memoria y procesa las instrucciones más rápido. Esto es vital cuando programamos robots que necesitan reaccionar en milisegundos ante cualquier imprevisto.

4. Argumentos Válidos y Demostraciones Formales

Cuando diseñamos cosas delicadas, como un equipo médico o un freno automático de un carro, no podemos dejar nada a la suerte. Tenemos que estar seguros de que el programa va a responder bien. El libro explica cómo armar argumentos válidos para garantizar que si las lecturas de los sensores son correctas, la respuesta de la máquina sea segura.

Para probar que nuestro código funciona bien antes de probarlo en la máquina real, usamos la demostración formal :

- Método directo : Es ir uniendo pasos lógicos uno detrás de otro para demostrar que, si las variables de entrada son las correctas, el programa va a llegar al resultado esperado.
- Demostración por contradicción : Aquí hacemos lo contrario. Suponemos temporalmente que la máquina va a fallar (por ejemplo, que un ascensor va a abrir las puertas mientras se está moviendo) y demostramos que matemáticamente esa falla es imposible con la lógica que escribimos. Si demostramos que la falla provoca una contradicción, significa que nuestro diseño es seguro.

5. Predicados e Inducción en Robótica Avanzada

Cuando trabajamos con sistemas más avanzados, como la visión artificial o robots que esquivan obstáculos, la lógica simple ya no nos alcanza. Ahí es donde entra la sección de predicados, que nos permite usar variables y cuantificadores como "para todo" ($\forall$) o "existe" ($\exists$).

Por ejemplo, si una cámara está revisando piezas en una fábrica, usamos esta lógica para analizar conjuntos de datos:

- Para todas las piezas ($\forall x$), su tamaño debe ser el correcto.
- Existe al menos un obstáculo ($\exists y$) en la trayectoria del robot (por lo que debe frenar).

Finalmente, la inducción matemática nos da la base teórica para entender y programar los bucles (for, while) y los procesos repetitivos que el procesador ejecuta una y otra vez para leer las señales analógicas del entorno.

Conclusión

Como conclusión, y tal como lo resume el libro de Jiménez Murillo, la aplicación de la lógica matemática no es solo teoría para pasar la materia. Para nosotros, los estudiantes de mecatrónica, es el idioma que nos permite conectar el mundo físico (voltajes, motores, sensores) con el mundo digital de la programación.

Aprender bien el Capítulo IV de *Matemáticas para la Computación* cambia la forma en que programamos: dejamos de escribir código probando a la suerte ("ensayo y error") y empezamos a diseñar algoritmos bien estructurados. Al final del día, esto es lo que asegura que las máquinas que automaticemos funcionen de forma exacta, eficiente y, sobre todo, segura.

Bibliografía

Jiménez Murillo, J. A. (2008). *Matemáticas para la computación* (1ra ed.). Alfaomega Grupo Editor.